BUG FIX

Filestore only stores the last block when `MaxMsgsPerSubject = 1`

A stale per-subject `fblk` pointer makes a single-message subject unreadable after restart — the `MaxMsgsPerSubject = 1` sibling of the FIFO bug in PR #8285.

PR: nats-io/nats-server#8254 **Files:** `filestore.go`, `memstore.go`

Affects: `MaxMsgsPerSubject = 1` (incl. default KV) **Trigger:** replace-on-publish + restart

---

Relationship to PR #8285. These two fixes are the same bug family — a lazily-updated per-subject first-block pointer (`fblk`) that becomes stale and gets persisted as the *only* pointer for a single-message subject. **#8254 (this report)** covers the `MaxMsgsPerSubject = 1` "replace the previous message" path. **#8285** covers the `MaxMsgsPerSubject > 1` FIFO-eviction path. Together they close both ways a subject can be trimmed down to one surviving message in a later block.

TL;DR

For each subject the filestore keeps `fblk` (first block) and `lblk` (last block) so lookups scan only the relevant block range. When `MaxMsgsPerSubject = 1`, publishing a new message for a subject **replaces** the previous one: the new copy lands in the current (later) block and the old copy is removed. The code updated `lblk` to the new block but left `fblk` pointing at the **old, now-empty** block. Because a single-message subject persists **only** `fblk` to `index.db`, that stale value survived a restart and the message became unreadable. PR #8254 adds a one-line correction — when a subject is down to one message under a per-subject limit of 1, set `fblk = lblk` — plus a related fix so switching a stream to a per-subject limit actually enforces it.

Background: `fblk` / `lblk` and how single-message subjects persist

A file-backed stream is split into many on-disk blocks (`1.blk`, `2.blk`, …). Per subject the store keeps a small record:

```
type psi struct {
    total uint64  // messages currently stored for this subject
    fblk  uint32  // first block holding a message for this subject
    lblk  uint32  // last block holding a message for this subject
}
```

Lookups use `[fblk .. lblk]` as a search window. The critical detail (shared with #8285): when stream state is snapshotted to `index.db`, a subject with `total == 1` persists only `total` and `fblk`; on restore `lblk` is reconstructed as `lblk = fblk`. **So for a one-message subject, `fblk` is the single source of truth — if it is wrong, both pointers are wrong after a restart.**

The bug: replace-on-publish leaves `fblk` behind

With `MaxMsgsPerSubject = 1`, the store keeps exactly one message per subject. On each new publish for an existing subject, the new record is appended to the current block and the previous record is removed by the per-subject limit. In the store path, `lblk` is advanced to the new block…

```
// filestore.go — storeRawMsg, per-subject bookkeeping
if info, ok = fs.psim.Find(stringToBytes(subj)); ok {
    info.total++
    if index > info.lblk {
        info.lblk = index      // lblk advances to the new (current) block
    }
}
```

…but `fblk` is updated only *lazily* (normally corrected during a later lookup). If the old copy is removed and the state is snapshotted — then the server restarts — before any lookup runs, `fblk` still points at the old block, and that stale value is the only one persisted.

Walkthrough (`MaxMsgsPerSubject = 1`)

1. Subject `foo` has its one message in block 1. psi = {total: 1, fblk: 1, lblk: 1}

2. A new message for `foo` is published; it lands in the current block 3. `lblk` advances to 3. The per-subject limit (1) removes the old copy in block 1. `fblk` is **not** recomputed (lazy).

psi = {total: 1, fblk: 1 (stale!), lblk: 3}

red = `fblk` · green = `lblk`

3. State is snapshotted to `index.db`. Because `total == 1`, only `fblk = 1` is written.

4. Server restarts → `fblk = 1`, `lblk = fblk = 1`. The search window collapses onto block 1.

5. **A lookup for `foo` scans only block 1, finds nothing, and the live message in block 3 is invisible.**

The fix (PR #8254)

1. Correct `fblk` for single-message subjects

After the per-subject bookkeeping, if the subject is down to a single message under a per-subject limit of 1, eagerly snap `fblk` to `lblk` so the persisted pointer references where the message actually is:

```
// If we only ever store one/last message for a subject,
// can correct the first block to where we've just written.
if info != nil && info.total == 1 && mmp == 1 {
    info.fblk = info.lblk
}
```

Now at step 2 above, `fblk` becomes 3, the persisted pointer is correct, and after restart the lookup finds the message. (This is the exact correction PR #8285 later restored/extended to the

`MaxMsgsPer > 1` FIFO path on the 2.12 line.)

2. Actually enforce the limit when config switches to per-subject

The condition that decides whether to run per-subject enforcement on a config update was tightened from `== 0` to `≤ 0`, so a transition from *unlimited* (`-1`) to a positive limit is correctly treated as "newly limited" and triggers a trim:

```
// filestore.go — on UpdateConfig
if fs.cfg.MaxMsgsPer > 0 && (old_cfg.MaxMsgsPer <= 0 || fs.cfg.MaxMsgsPer < old_cfg.MaxMsgsP
    fs.enforceMsgPerSubjectLimit(true)
}
```

Before the fix, `old == 0` would be false when the previous value was `-1` (unlimited), so enabling a per-subject limit on an existing stream would silently skip enforcement, leaving over-limit subjects untrimmed (and, combined with issue #1, prone to the stale-pointer problem).

3. Memstore parity

The same `≤ 0` enforcement-trigger condition was mirrored in `memstore.go` for consistency. (The stale-`fblk` persistence problem itself is filestore-only, since memory stores have no `index.db` and do not survive a restart.)

```
// memstore.go
if ms.maxp > 0 && (maxp <= 0 || ms.maxp < maxp) {
    // enforce new per-subject limit
}
```

Tests

Added coverage in `server/filestore_test.go` and `server/store_test.go` verifying that a message stays retrievable after a config update to `MaxMsgsPerSubject = 1` and after a server restart — i.e. that `fblk` is no longer stale.

Which configurations are affected

Two conditions must hold:

1. **File storage** — the bug only manifests across a restart via `index.db`; memory streams are immune.
2. **`MaxMsgsPerSubject = 1`** — the replace-on-publish path that strands `fblk` when a subject's single message moves to a newer block.

A secondary exposure: enabling a per-subject limit on a stream that previously had none (`-1` → positive) did not enforce the limit before the fix.

Examples that ARE affected

```
{
  "name": "LATEST",
  "storage": "file",
  "subjects": ["state.>"],
  "max_msgs_per_subject": 1     // keep only the latest value per subject
}
```

"Last value cache" / latest-state streams are the classic shape here: one retained message per subject, overwritten on each publish.

KV buckets — the default is affected

A KV bucket maps to a stream with `MaxMsgsPerSubject = history`, and the **default history is 1**. So an ordinary file-backed KV bucket created without an explicit history sits exactly on this bug:

```
nats kv add CONFIG --storage=file        // history defaults to 1 -> affected by #8254
nats kv add CONFIG --storage=file --history=5  // history > 1 -> affected by #8285 ins
```

After a restart, an affected key can appear **missing** on `kv.Get` even though the value is intact on disk, and conditional writes mis-evaluate (see next section). Note this is the mirror image of #8285: **#8254 hits `history = 1` (the default), #8285 hits `history > 1`** — between them, KV buckets at any history were exposed on affected releases.

Not affected

- **Memory storage** — no `index.db`, no restart-restore path.
- **`MaxMsgsPerSubject > 1`** — a different path (covered by PR #8285), not this fix.

- **No per-subject limit at all** (`-1`) — nothing trims the subject to a single stale-pointer message.

Knock-on effect: `Nats-Expected-Last-Subject-Sequence`

As with #8285, the damage isn't limited to plain reads. The per-subject optimistic-concurrency header is evaluated via `store.LoadLastMsg(subject)` → `loadLast`, which scans backwards over the same `[lblk .. fblk]` window:

```
// stream.go
if seq, exists := getExpectedLastSeqPerSubject(hdr); exists {
    sm, err := store.LoadLastMsg(seqSubj, &smv)   // uses fblk/lblk
    if sm != nil { fseq = sm.seq }
    if err != nil || fseq != seq {              // mismatch -> reject publish
        resp.Error = NewJSStreamWrongLastSequenceError(fseq)
    }
}
```

With a stale `fblk`, `LoadLastMsg` returns "not found" (`fseq = 0`) for a subject that does have a message. That corrupts the check both ways: a valid conditional publish at the real sequence is **wrongly rejected**, while a create-only publish (expected sequence `0`) is **wrongly accepted** and clobbers the existing value. For KV (`history = 1`) this surfaces as broken `Update`/`Create` (CAS) semantics after a restart.

Version comparison

Behavior	Pre-fix	With PR #8254
Corrects `fblk` when subject → 1 msg under `MaxMsgsPer = 1`	No	Yes
Single-msg subject retrievable after restart (`MaxMsgsPer = 1`)	No — bug	Yes
Enforces limit when config switches `-1` → positive	No (`== 0` guard)	Yes (`≤ 0` guard)
`Nats-Expected-Last-Subject-Sequence` correct after restart	No — bug	Yes

Behavior	Pre-fix	With PR #8254
Memory store enforcement parity	Inconsistent	Aligned

Note: 2.14+ also persists both `fblk` and `lblk` to `index.db` unconditionally, which independently neutralizes the stale-pointer class of bug regardless of `MaxMsgsPerSubject`. PR #8254 is the targeted source-level fix; the format change is the structural one.

Recovery for already-affected data

Identical to the #8285 remediation: the stale pointers live only in `index.db`; the messages are intact in the block files. Force a rebuild:

1. **Stop the nats-server** (it rewrites `index.db` on clean shutdown).
2. **Delete only `index.db`** for the affected stream:

   ```
   rm <store_dir>/jetstream/<account>/streams/<stream>/msgs/index.db
   # e.g. KV bucket CONFIG -> stream KV_CONFIG
   ```

   Leave all `*.blk` files in place.
3. **Restart.** The server rebuilds `fblk` / `lblk` by scanning the blocks; the “missing” message reappears and the expected-last-subject-sequence check returns the correct value.
4. **Clustered (R>1):** recover one replica at a time, preserving quorum; or step down / re-add the replica to re-sync from the leader.

> The rebuild is safe and non-destructive (`index.db` is always reconstructable from the blocks), but scans the whole stream — expect extra startup I/O for large streams. To *prevent* recurrence, upgrade to a build that includes PR #8254 (and #8285 for the `MaxMsgsPer>1` path), or to 2.14+.

Scope & limitations

- **Targets the `MaxMsgsPerSubject = 1` path.** The `MaxMsgsPer>1` FIFO-removal variant is handled separately by PR #8285.
- **File storage only** for the data-recovery aspect; the enforcement-condition fix also applies to memstore.
- **Low risk.** A one-line pointer correction plus a guard-condition broadening, with added regression tests; no on-disk format change.

Generated explainer for nats-io/nats-server#8254 (sibling of #8285). Code references: `server/filestore.go` (`storeRawMsg` per-subject bookkeeping, `enforceMsgPerSubjectLimit`, `loadLast`), `server/memstore.go`, `server/stream.go` (expected-last-subject-sequence). For authoritative behavior, consult the PR and the source.